

PvrGPU C-Model 加入 Ray Tracing Stage：分階段方案

以 PowerVR Photon 架構與 Vulkan/DXR Acceleration-Structure + SBT 模型為對照

專案：PvrGPU (Mesa Gallium PCO Driver + SystemC 功能模擬 C-Model)

範圍：src/systemc/common、新增 RT 相關 module，以及 driver 端 dispatch 介面

日期：2026-09-04 撰寫依據：對 repo 快照的直接原始碼查證，非臆測

本文規劃在 PvrGPU C-Model 現有的光柵化管線之外，加入一條 ray tracing 路徑。方案分兩部分：先說明這條路徑如何接進現有的 module/PipelineState/MemoryMode 慣例（含一個目前無法確認的缺口），再給出六個階段的實作拆分與預估影響檔案。全文分析基於某一時間點的 repo 快照，另有其他 agent 正在同一 repo 上進行 GFXBench/Manhattan 相關修正；本方案刻意避開 pvrgpu_context.c，可與該修正並行推進。

一、如何接進現有架構

1.1 Pipeline stage 是集中列舉的

PipelineStage enum 定義在 common/functional_types.h（並非 pipeline_state.h），從 kSubmitted 一路列到 kFramebufferReady，每個 module 進入 Run() 時都會呼叫 RequireStage() 做 fail-closed 檢查。加入 RT 需要在此 enum 插入新值（例如 kAccelStructureBuilt、kRayTraced），這是全 model 唯一必須觸碰「共用列舉」的地方，其餘既有 stage 完全不受影響。

1.2 Module 接線：直線 FIFO chain，但已有側邊 port 先例

所有 module 都是 sc_fifo_in<PipelineTxn> input / sc_fifo_out<PipelineTxn> output 的標準殼，用一個 PoolHandle 帶著序列化的 PipelineState 在 module 間傳遞。UscCluster 額外開了 SC_ZERO_OR_MORE_BOUND 的側邊 port (texture_request_output / texture_response_input) 供 texture 流量使用——這正是 RT traversal 單元需要的模式：BVH 節點讀取要頻繁戳 memory / TCU，不應擠進主 chain，應仿照 USC 開側邊 port 直接對 TCU/SLC。

1.3 PoolHandle：index/generation handle，不是指標

PipelineState 已有一批 PoolHandle 欄位 (vertex buffer、code、tile record……)，要加 PoolHandle acceleration_structure 與 PoolHandle shader_binding_table 屬於同一種加法，跟先前為線 / 點光柵化加入 source_topology / raster_state 是同一手法。設計文件 (PvrGPU.md) 中已描述目標形態 TxnRef{handle, offset, bytes, ...}，支援切片 / view-only handle，適合裝載較大塊的 BVH 節點資料而不必整包複製——但這是文件中的目標方向，目前實作的 MemoryPool 僅有基本的 Allocate / Read / Write，細粒度切片尚未落地。

1.4 MemoryMode 開關可直接複製給 RT 用

現有鏈路：pvrgpu_cmd.c 讀環境變數 PVRGPU_MODEL_MEMORY_MODE → 填入 pvrgpu_systemc_api.h 的 struct pvrgpu_systemc_submit_info（目前是 const char *memory_mode 欄位）→ 經 dlopen / dlsym 的 ABI 邊界傳給獨立建置的 SystemC 共享函式庫。RT on/off 開關可以照抄這條路徑，新增例如 PVRGPU_MODEL_RT_ENABLE 環境變數與對應欄位。這條路徑完全不會碰到 pvrgpu_context.c，與目前另一併發 agent 的工作範圍沒有重疊，方案可以現在就開始。

1.5 USC 可重用，但 ShaderStage 需先擴充

PcoDecoder/UscSlot/UscCluster 三件套是以 ShaderStage 參數化實例化的（vertex/fragment 各開一份 instance），結構上對新 stage 是可直接複製的樣板。但 ShaderStage enum 目前只有 kVertex/kFragment 兩個值（連 kCompute 都還沒有），pco_iss.h 的 PcoVertexExecution/PcoVertexContinuation 與 PcoFragmentExecution/PcoFragmentContinuation 也是各自獨立的 struct，並非通用的 shader-invocation 抽象。要讓 raygen/closest-hit/any-hit/miss shader 跑在同一個 USC/PCO ISS 上，做法是比照現有模式：ShaderStage 加新值、每個 RT shader 類型各開一份 PcoDecoder+UscSlot+UscCluster instance、pco_iss.h 仿照 PcoVertexExecution 那組加一組 PcoRayExecution/PcoRayContinuation。這是「照既有模式複製」而非「發明新機制」，但涉及檔案不少，工作量不可低估。

缺口說明：頂層 module 接線檔（sc_main、負責 .bind() 全部約 21 個 instance 的獨立 source）在目前可讀取的檔案集裡沒有找到；設計文件說明「top-level 必須放在獨立 source，不得定義 module class」，但實際檔案不在快照範圍內。實際插入 RT stage 的接線動作（第五節 Stage 2、3）需要這個檔案才能給出精確 diff，建議鎖解開、能重新取得完整 repo 之後，第一件事就是核對這個檔案，再開始寫接線程式碼。這不影響 Stage 0/1（純資料結構與獨立功能模組）現在就動工。

二、API 語意對照 (Vulkan/DXR)

Vulkan VK_KHR_ray_tracing_pipeline/VK_KHR_acceleration_structure 與 DXR 的模型一致：應用程式先用 vkCmdBuildAccelerationStructuresKHR 建出 Bottom-Level (幾何, BLAS) 與 Top-Level (實體/instance transform, TLAS) 加速結構；再準備一份 Shader Binding Table (SBT)，把 ray hit group 映射到對應的 raygen/closest-hit/any-hit/miss shader；最後用 vkCmdTraceRaysKHR/DispatchRays 送出 dispatch。Gallium 的 pipe_context/pipe_screen 介面並不承載 ray tracing——真實的 Vulkan driver 通常繞過 Gallium 的抽象層，直接在 driver 層實作 AS 建構與 SBT dispatch。這一點決定了本方案第四節 Stage 4 的介面選擇：走一條獨立的 driver-model API 通道 (比照 memory_mode 的做法)，而不是硬塞進既有 pipe_context 介面。

PowerVR Photon 是本方案主要參考的公開架構：在既有 TBDR 管線 (VDM/Tiler/ISP) 之外加入一顆專用的 BVH traversal 單元，第五節 Stage 2 的 RayTraversalUnit 即對應這個角色。

三、分階段實作計畫

設計原則與先前的 scheduling 方案一致：每階段附加式 (additive)、向下相容——RT 關閉時，既有光柵管線行為必須與今天完全一致。Stage 0–3 全部發生在 common/、新增的 systemc/module 檔案裡，與目前卡住的 pvr gpu_context.c、以及尚待合併的線/點光柵化改動完全不重疊，可以現在就開始撰寫。

Stage 0 資料結構落地，不跑任何東西

在 PipelineState 加入 PoolHandle acceleration_structure/PoolHandle shader_binding_table，以及一個描述 BLAS/TLAS 幾何範圍的小 struct；在 PipelineStage enum 插入 kAccelStructureBuilt、kRayTraced 兩個新值 (順序上代表與光柵管線平行、於 shading 前匯合)。純加欄位，不改任何既有 module 邏輯，零回歸風險，可以立即動工。

Stage 1 BVH 建構的功能性 stub (先求正確，不求真時序)

新增獨立 module AccelStructureBuilder，輸入沿用 vertex fetch 之後的頂點/index buffer，輸出寫入 acceleration_structure handle。第一版走 host 端純函式建 BVH (比照 texture cache 先前的功能性路徑，先不接時序)，SystemC 只包一層 Run() 收發 PipelineTxn。這階段驗證的是資料格式 (BLAS/TLAS 層次、AABB 節點佈局) 是否正確，不涉及效能模型。

Stage 2 Traversal 硬體模型 (對照 Photon 白皮書核心)

新增 RayTraversalUnit module，職責對應 Photon 架構中的 BVH traversal 專用單元：從 SBT 找到對應 hit group、走 BVH 節點、與三角形做交集測試。需要側邊 port 頻繁戳記憶體讀取 BVH 節點，比照 UscCluster 的 SC_ZERO_OR_MORE_BOUND 側邊 port 模式接到 TCU/SLC，不走主 FIFO chain。先給簡化時序模型 (例如每節點固定 cycle 數 + memory 延遲)，細緻 latency 校準留待後續。此階段需要先核對頂層接線檔 (見第一節缺口說明) 才能定案精確的 port 綁定方式。

Stage 3 Shader 端擴充 (raygen/closest-hit/miss)

ShaderStage enum 加入 kRayGeneration、kClosestHit、kAnyHit、kMiss。pco_iss.h 加入對應的 PcoRayExecution/PcoRayContinuation/PcoRayExecutionContext，結構仿照 PcoVertexExecution 那組。每個新 stage 各開一份 PcoDecoder+UscSlot+UscCluster instance（與現有 VS/FS 兩份並列的做法一致）。RayTraversalUnit 命中後組出對應 PipelineTxn 丟給命中的 shader instance，miss 則丟給 miss shader instance。此階段依賴 Stage 2 的命中結果格式，但與 USC 是否有 multithreading 無關——與先前討論的 coarse/fine-grained scheduling 方案是正交的兩條線，互不阻塞、可平行推進。

Stage 4 SBT 驅動的 dispatch 語意 (對齊 Vulkan/DXR)

在 driver 端 (pvr gpu_pco.c 或新檔) 實作 SBT 建構邏輯：把 vkCmdBuildAccelerationStructuresKHR/vkCmdTraceRaysKHR 對應的語意，翻譯成 C-model 認得的 acceleration_structure/shader_binding_table handle，加上一個 dispatch 描述 (ray 數量、SBT stride/offset)。如第二節所述，這裡採用獨立的 driver-model API 通道 (比照 PVRGPU_MODEL_MEMORY_MODE 的做法)，不強行納入既有 pipe_context 介面。此階段是唯一可能需要接觸 driver 提交路徑 (而非 pvr gpu_context.c 本體) 的階段，建議排在鎖解除、GFXBench 修正穩定後再做。

Stage 5 開關與回歸驗證

比照 MemoryMode 前例，新增 RT on/off 開關 (例如 PVRGPU_MODEL_RT_ENABLE)。關閉時，Stage 0-4 新增的 module 與欄位完全不參與既有光柵管線，行為與今天一致。開啟時，跑一組新增的最小 dEQP-VK ray tracing case 驗證正確性，不動既有光柵 regression baseline (1849 pass/1 fail gl_manhattan)。

四、預估影響檔案

檔案	變動內容
common/functional_types.h	PipelineStage enum 新增 kAccelStructureBuilt、kRayTraced
common/pipeline_state.h	新增 acceleration_structure/shader_binding_table 等 PoolHandle 欄位
common/shader_stage.h	ShaderStage 新增 kRayGeneration/kClosestHit/kAnyHit/kMiss
systemc/accel/accel_structure_builder.* (新增)	Stage 1 : BVH 建構功能性 stub module
systemc/accel/ray_traversal_unit.* (新增)	Stage 2 : traversal 硬體模型，側邊 port 接 TCU/SLC
shader/pco_iss.h (及 .cpp)	新增 PcoRayExecution/PcoRayContinuation/PcoRayExecutionContext
shader/pco_decoder.*、usc_slot.*、usc_cluster.*	為 RT shader stage 各開一份 instance (沿用既有 per-stage 模式)
頂層接線檔 (sc_main/module 拓樸，檔名待核對)	Stage 2、3 : 新 module 的 FIFO/側邊 port 綁定

gallium/drivers/pvrgpu/pvrgpu_pco.c (或新檔)	Stage 4 : SBT 建構、AS build / trace-rays dispatch 語意轉譯
gallium/drivers/pvrgpu/pvrgpu_cmd.c、pvr gpu_systemc_api.h	Stage 5 : 新增 RT on/off 環境變數與傳遞
common/pipeline_state.h (CounterTxn)	新增 RT 相關計數器 (traversal 節點數、hit/miss 次數等)

結論：Stage 0–3 風險偏低、屬於附加式改動，可立即開始且與目前併發修正、既有線／點光柵化改動完全不重疊。Stage 4 (driver 端 dispatch 語意) 與頂層接線檔的核對是主要風險與不確定性來源，建議排在鎖解除之後再做，且動工前先重新核對本文提到的所有行號與檔案是否已因後續 commit 偏移。

五、與現有工作的關係

本方案與先前提出的 texture L2 cache、USC coarse/fine-grained scheduling 兩項提案彼此正交、互不阻塞：RT traversal 對記憶體存取模式（不規則、指標追逐式的 BVH 節點讀取）與 texture 取樣的存取模式差異很大，不建議直接共用同一顆 L2；但若 scheduling 方案的 Stage 2（常駐 context + coarse-grained switch-on-stall）先落地，RT shader stage 的 raygen/hit/miss instance 也能直接受益於同一套「等待記憶體時切換 context」機制，不需要額外設計。

建議順序：可與 texture L2、scheduling 方案的 Stage 0 同時開始撰寫（三者互不衝突）；RT 的 Stage 4（driver dispatch 語意）與 scheduling 的 Stage 4（driver 端切換開關）則都建議排在 git/build 鎖解除、GFXBench 修正合併之後，避免與併發 agent 的驗證流程互相干擾。

本文所有檔案路徑、行號與現況描述，均為對某一時間點 repo 快照直接讀碼所得，非推測；第一節已明確標出目前無法確認的缺口（頂層接線檔）。實作前建議重新核對 live repo（尤其是另一 agent 仍在進行中的變動），以及本文引用行號在後續 commit 後是否已偏移。